

Here is a clean handoff summary you can paste into a new chat.

Handoff Summary: AIAIchemy Governed Multi-Agent Intelligence Stack

Overall Objective

I want to build one integrated system, not a set of disconnected AI projects.

The system should be a Streamlit-based governed multi-agent intelligence stack that can:

use synthetic structured data

use a synthetic document database

optionally pull from my existing `contractriskreviewpipeline`

run document review

run advanced data analysis

run projections/forecasting, including Fourier-style seasonality projections

generate charts and reports

show governance around every agent, skill, decision, approval, failure, and output

The portfolio story is:

> "I built a governed multi-agent intelligence stack that can review documents, analyze data, forecast trends, generate reports, and prove every agent action was controlled, logged, verified, and recoverable."

Existing Repositories / Prior Work

Relevant existing repos include:

`AIAIchemy-Repo-Governor`

Should become the repo safety / harness readiness layer.

Scans repos, classifies risk, generates agent policies, checks readiness.

`ai-agent-governance`

Should become the control plane.

Handles agent registry, approvals, promotion, rollback, risk governance, and audit trails.

`governance-logger`

Should become the audit/observability layer.

Logs agent actions, success, errors, tests, retries, and governance events.

`Agent-Workflow-Review`

Should become the pre-run review board.

Risk agent, value agent, decision synthesizer.

`workeragentcowork`

Useful pattern for planner-worker orchestration, verification, SQLite storage, and reporting.

`contractriskreviewpipeline`

Should be treated as an optional existing data/document source through an adapter.

Do not hardwire the new stack to it.

Main architectural decision:

> Build a new master repo, likely called `ai-alchemy-agent-stack`, and reuse/refactor code from the existing repos as modules.

Core Build Concept

The system should be one Streamlit app with tabs and dropdowns.

The user should be able to select:

Data source

Synthetic structured database

Synthetic document database
Existing contract risk pipeline
Hybrid: synthetic + existing
Workflow type
Contract/document review
Risk analysis
Financial/data projection
Compliance review
Executive report
Governance audit report

The system should not rely primarily on uploaded documents. It should come with built-in synthetic data and synthetic documents so the stack can be safely demonstrated.

Key Architecture

Recommended folder structure:

```text

ai-alchemy-agent-stack/

  app.py

  orchestrator/

    orchestrator.py

    planner.py

    router.py

    message\\_schema.py

  agents/

    document\\_agent.py

    data\\_analyst\\_agent.py

    forecast\\_agent.py

    report\\_agent.py

    verifier\\_agent.py

    governance\\_arbiter.py

  skills/

    registry.yaml

    executor.py

    models.py

  builtins/

    query\\_structured\\_data.py

    retrieve\\_document.py

    review\\_document.py

    extract\\_risks.py

    run\\_eda.py

    run\\_regression.py

    run\\_fourier\\_forecast.py

    generate\\_chart.py

    generate\\_report.py

    verify\\_report.py

governance/  
    workflow\\_charter.py  
    policy\\_engine.py  
    gate\\_engine.py  
    audit\\_logger.py  
    rollback\\_manager.py  
    recovery\\_manager.py  
    cost\\_tracker.py  
    trust\\_engine.py  
    security\\_engine.py  
    failure\\_taxonomy.py  
    authority\\_model.py  
    evidence\\_model.py  
    escalation\\_policy.py  
    evaluation\\_plan.py  
    monitoring\\_plan.py  
    accountability\\_model.py

data\\_sources/  
    synthetic\\_data.py  
    synthetic\\_documents.py  
    contractrisk\\_adapter.py  
    hybrid\\_adapter.py

analytics/  
    cleaning.py  
    eda.py  
    regression.py  
    fourier.py  
    forecasting.py  
    projections.py

reporting/  
    report\\_writer.py  
    chart\\_builder.py  
    word\\_report.py  
    powerpoint\\_report.py  
    pdf\\_report.py

memory/  
    working\\_memory.py  
    session\\_store.py  
    long\\_term\\_store.py  
    quarantine\\_store.py

storage/  
    synthetic\\_db.sqlite  
    skill\\_runs.sqlite  
    audit\\_log.jsonl  
    checkpoints/

```

reports/

tests/
 test_skills.py
 test_policies.py
 test_gates.py
 test_orchestrator.py
 test_forecasting.py

AGENTS.md
CLAUDE.md
repo_policy.yaml
README.md
```
---
Core Layers
1. Workflow Charter Layer
This is mandatory. No charter, no run.
The workflow charter captures:
Business problem
Desired outcome
Decision environment
Users/stakeholders
Agent role
Boundaries
Authority
Data access
Tool access
Risk level
Human approval points
Evidence requirements
Failure modes
Evaluations
Monitoring
Governance lifecycle
Accountability owner
Key idea:
> Boundaries are not enough. The system also needs authority, evidence, escalation,
testing, and accountability.
This turns the system from “AI doing tasks” into a governed operating model.
---
2. Data Source Layer
The system should support:
synthetic structured data
synthetic document corpus
optional existing contract-risk-review pipeline data
hybrid mode
Each source should output standard internal records:
```text
DocumentRecord

```

StructuredDataRecord  
RiskFinding  
AnalysisResult  
ForecastResult  
ReportArtifact  
AuditEvent  
...

The app should not care where the data came from.

---

### 3. Agent Layer

Use narrow agents, not one giant agent.

Recommended agents:

```text

PlannerAgent
RouterAgent
DocumentReviewAgent
DataAnalystAgent
ForecastAgent
ReportWriterAgent
VerifierAgent
GovernanceArbiterAgent
...

Important rule:

> Agents should not freely talk to each other. All coordination should go through the orchestrator.

Good pattern:

```text

Agent → Orchestrator → Agent

...

Bad pattern:

```text

Agent → Agent → Agent → Agent

...

4. Orchestration Layer

The orchestrator is the traffic controller.

It should handle:

task decomposition
agent selection
skill routing
context passing
loop detection
delegation depth limits
fan-out/fan-in coordination
critic/verifier routing
failure escalation

5. Skills-Led Governance Layer

Every agent action should be treated as a governed skill call.

Example skills:

```
```text
query_structured_data
retrieve_synthetic_document
retrieve_contract_pipeline_document
review_document
extract_risks
run_eda
run_regression
run_fourier_forecast
generate_chart
generate_report
verify_report
log_audit_event
```
```

Each skill should have:

```
skill ID
semantic version
allowed agents
allowed roles
input schema
output schema
risk tier
policy envelope
gate requirements
cost budget
rollback behavior
audit requirements
trust requirements
```

6. Policy Layer

The policy layer answers:

Is this agent allowed to use this skill?

Is this data allowed?

Is this action allowed in the current environment?

Is approval required?

Should output be redacted?

Should the run be blocked?

Policy categories:

```
```text
ExecutionPolicy
AuthorizationPolicy
DataPolicy
EnvironmentPolicy
CostPolicy
RetentionPolicy
```
```

7. Gate Layer

Gates are checkpoints before and after skill execution.

Gate types:

```text

PreExecutionGate  
HumanApprovalGate  
CostGate  
QuotaGate  
DuplicateDetectionGate  
OutputValidationGate  
SecurityGate  
DownstreamGate  
```

The app should include intentional failure demos:

```text

Bad input demo  
Quota failure demo  
High-risk approval demo  
Duplicate call demo  
Cost-limit demo  
Untrusted agent demo  
Rollback demo  
```

Purpose: make governance visible.

8. Analytics and Forecasting Layer

The stack should perform real analysis:

cleaning

EDA

descriptive statistics

correlation analysis

regression

Fourier features

seasonality modeling

scenario projection

charts

plain-English explanations

Fourier-style projections should be used to demonstrate seasonality/cyclical patterns, not as magic prediction.

9. Reporting Layer

Reports should include:

executive summary

key findings

risks

charts

forecasts

recommendations

evidence

governance notes

limitations

Output formats may include:

```text

Markdown  
HTML  
DOCX  
PDF  
PPTX  
CSV  
PNG charts  
...

MVP can start with Markdown/HTML and later add Word/PDF/PPTX.

---

## 10. Memory Layer

Memory should be governed.

Memory tiers:

```text

WorkingMemory

SessionMemory

LongTermMemory

QuarantineMemory

```

Critical rule:

> Validate before memory write.

Do not let bad agent output contaminate shared memory.

---

## 11. Audit and Observability Layer

Every workflow should show:

who started it

which agent ran

which skill ran

what policy applied

which gates passed/failed

approval status

cost estimate

runtime

output hash

input hash

failure class

parent/child task relationship

Suggested audit fields:

```text

run_id

task_id

agent_id

skill_id

skill_version

input_hash

output_hash

policy_decision

gate_result

approval_status

duration_ms

estimated_cost
failure_class
timestamp
parent_task_id
```\n

---

## 12. Rollback and Recovery Layer

The system should support:

retry with backoff  
fallback skill  
checkpoint resume  
rollback where possible  
dead letter queue  
quarantine failed output  
human escalation

---

## 13. Security Layer

The system should protect against:

prompt injection  
treating document text as instructions  
privilege escalation  
unsafe tools  
sensitive data leakage  
untrusted inter-agent messages

Key rule:

> External content is data, not instruction.

Controls:

```text

Input sanitization
Instruction/data separation
Prompt injection detection
Permission ceiling enforcement
Output filtering
Sensitive-data redaction
Tool allowlist
Skill definition hash verification
```\n

---

## 14. Failure Taxonomy

Use standard failure classes:

```text

SKILL_FAILURE
AGENT_FAILURE
ORCHESTRATION_FAILURE
MEMORY_FAILURE
GOVERNANCE_FAILURE
SECURITY_FAILURE
CASCADE_FAILURE
DATA_SOURCE_FAILURE
REPORT_GENERATION_FAILURE

FORECASTING_FAILURE
```

Map each to:

```text

retry

fallback

halt

escalate

rollback

quarantine

alert

```

---

15. Human Oversight Interface

Streamlit should include:

workflow charter tab

data source selector

workflow runner

plan approval

approval queue

emergency halt

policy violation viewer

skill registry viewer

gate results

audit log browser

memory/checkpoint viewer

cost tracker

report download panel

Human actions should include:

```text

approve

reject

pause

rerun

halt

override with justification

inspect state

download report

```

---

16. Compliance and Evidence Layer

The system should be able to prove governance happened.

Evidence outputs:

```text

PolicyCoverageReport

SkillCoverageReport

AuditEvidenceReport

IncidentReport

GovernanceRunSummary

```

This is important for the portfolio story.

---

MVP Workflow 1: Structured Data + Forecasting

```text

Source selection

↓

Workflow Charter

↓

PlannerAgent creates workflow plan

↓

GovernanceArbiter reviews plan

↓

query_structured_data skill

↓

run_eda skill

↓

run_fourier_forecast skill

↓

generate_chart skill

↓

generate_report skill

↓

verify_report skill

↓

audit log written

↓

report downloadable

```

---

MVP Workflow 2: Synthetic Document Review

```text

Source selection

↓

Workflow Charter

↓

retrieve_document skill

↓

review_document skill

↓

extract_risks skill

↓

generate_report skill

↓

verify_report skill

↓

audit log written

↓

report downloadable

```

---

## Build Phases

### Phase 1: Foundation

Build:

```text

Streamlit UI
Synthetic structured database
Synthetic document database
Source dropdown
Basic workflow runner
Basic audit log
```

### Phase 2: Skills Governance

Build:

```text

Skill registry
Skill executor
Policy engine
Gate engine
Skill run logging
Intentional failure demos
```

### Phase 3: Agents and Orchestration

Build:

```text

PlannerAgent
RouterAgent
DocumentAgent
DataAnalystAgent
ForecastAgent
ReportAgent
VerifierAgent
GovernanceArbiter
```

### Phase 4: Analytics and Reporting

Build:

```text

EDA
Regression
Fourier forecast
Chart generation
Markdown report
Word/PDF/PPT export later
Report verification
```

### Phase 5: Memory and Recovery

Build:

```text

Session memory
Checkpoints
Resume from failure

Rollback demos
Dead letter queue
Quarantine memory
```

#### Phase 6: Advanced Governance

Build:

```text

Cost governance
Trust levels
Security checks
Failure taxonomy
Human approval queue
Compliance evidence reports
```

---

#### What Not To Do

Do not:

build ten separate apps  
let agents call each other freely  
make one giant agent do everything  
skip audit logs  
skip verification  
overbuild compliance before the demo works  
make the system dependent on real client data  
ask Codex/Claude Code to build the whole thing in one prompt  
rely on uploads as the primary workflow  
hardwire the system to `contractriskreviewpipeline`

---

#### Best Next Step

Create a new master repo:

```text

ai-alchemy-agent-stack
```

Then build Phase 1 only:

Streamlit app  
synthetic structured database  
synthetic document database  
source dropdown  
workflow charter tab  
basic workflow runner  
basic audit log

After that, add skill registry and policy gates.